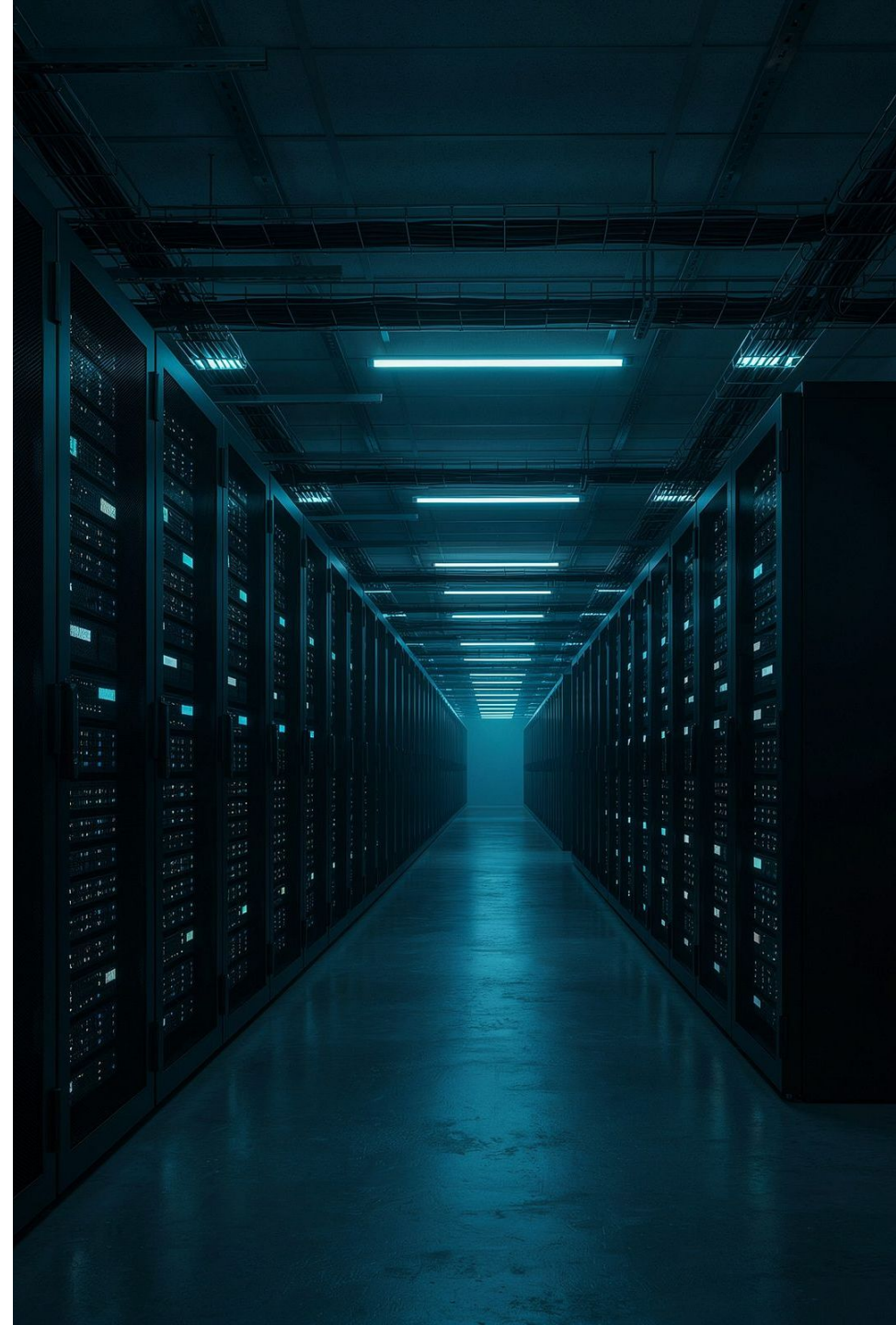


Day 5 — Operations, Troubleshooting & Mini Capstone

Jenkins CI/CD · Final Session



Learning Objectives

By the end of this session, you will be able to:

Operations & Troubleshooting

- Read Jenkins Pipeline logs
- Identify failed Pipeline stages
- Troubleshoot common Jenkins failures
- Diagnose credential and SCM issues
- Identify basic Agent-related problems
- Understand deployment failure scenarios

Capstone

- Build an end-to-end CI/CD Pipeline
- Intentionally introduce and diagnose a failure
- Fix and rerun a failed Pipeline

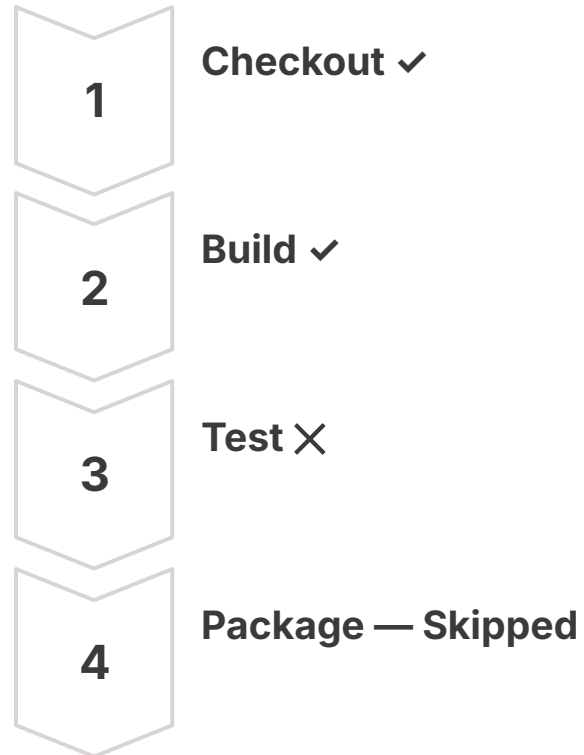
📄 Git → Build → Test → Artifactory → Approval → Deploy → Troubleshoot



SECTION 1

When a Pipeline Fails

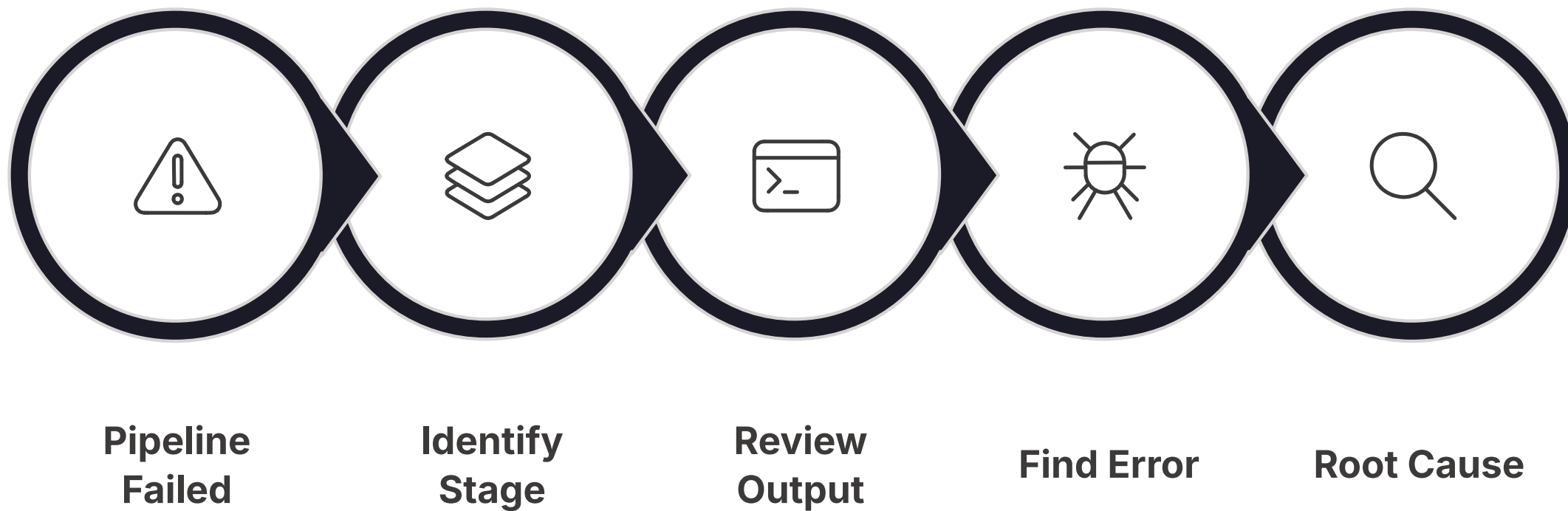
A CI/CD Pipeline contains multiple dependent activities. A failure at any stage halts the stages that follow.



📄 First question: **Which stage failed, and why?**

A Structured Troubleshooting Approach

Avoid making random changes when a Pipeline fails. Start with evidence from the Pipeline execution.



Reading Pipeline Logs

Jenkins Console Output provides execution details for every build. Look for the **first meaningful error**—not simply the final FAILURE message.

Logs Can Show

- **Commands executed & stage execution**
- **Tool output & test failures**
- **Authentication & SCM errors**
- **Build-tool & deployment errors**

Example Console Output

```
[Pipeline] stage  
[Pipeline] { (Build)  
$ mvn clean package  
[ERROR] COMPILATION ERROR  
[ERROR] BUILD FAILURE  
[Pipeline] }
```

SECTION 1

Understanding Failed Stages

Pipeline visualization helps identify exactly where execution stopped. Use both **stage status** and **Console Output** when troubleshooting.

1

Which stage failed?

2

Which command was
executing?

3

What does the log say?

4

Application, config, infrastructure, or access?

5

Were later stages skipped?

SCM Issues

Typical Symptoms

- Repository not found
- Authentication failed
- Couldn't find remote ref
- Connection timed out

Areas to Check

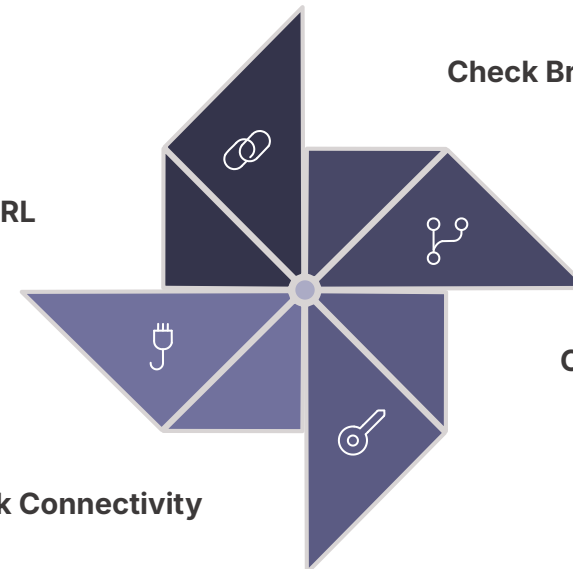
- Repository URL & branch name
- Git configuration & credentials
- Repository permissions
- Network connectivity

Check Repo URL

Check Branch

Check Credentials

Check Connectivity



Credential Issues

Typical Symptoms

- 401 Unauthorized
- 403 Forbidden
- Authentication failed
- Permission denied

What to Check

- Correct Credential ID and type
- Username/password or token validity
- Credential permissions & external-system access

Never Do This

Do not print passwords to the console to debug. Never expose secrets in logs while troubleshooting.

✓ Better Approach

Verify the **credential configuration and access**, not the secret value itself.

Agent Issues

A Pipeline may fail because the required execution environment is unavailable. If no available Agent has the required label, the Pipeline will wait indefinitely.

Example

```
agent { label 'linux' }
```

If no Agent carries the `linux` label, the Pipeline stalls in the build queue.

Areas to Check

- Agent online/offline status
- Node availability & executors
- Agent labels
- Required tools & workspace
- Connectivity



SECTION 2 — COMMON PROBLEMS

Build & Test Failures

Not every Jenkins failure is a Jenkins problem. Jenkins may simply be reporting a failure produced by another tool.

Build Failure

Jenkins → Maven → Compilation Error ✕

Possible cause: **Application source code**

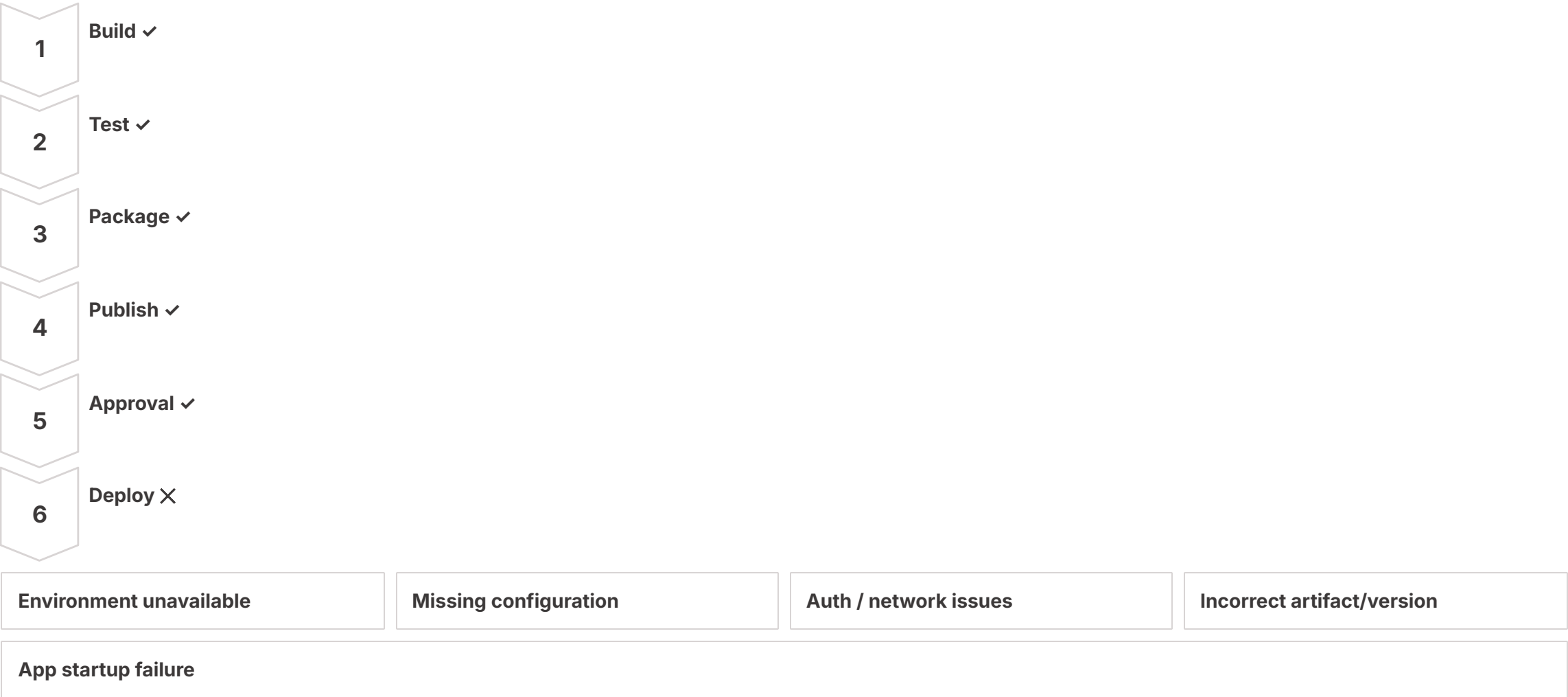
Test Failure

Build ✓ → Tests → 1 Test Failed ✕

Possible cause: **Application behavior or test expectation**

Deployment Failure Awareness

A successful build does not guarantee successful deployment. Day 5 focuses on **failure awareness and troubleshooting**, rather than deep production deployment engineering.



Mini Capstone Scenario

Build an integrated CI/CD Pipeline using concepts learned throughout the course.



Source Control

Git repository as the single source of truth for code and Jenkinsfile.



Build & Test

Maven compiles, runs unit tests, and packages the application.



Publish & Approve

Artifact published to JFrog Artifactory, followed by a manual approval gate.



Simulated Deploy

Approved artifact proceeds to a simulated deployment stage.



Capstone Jenkinsfile

The target Pipeline below covers all seven stages. Replace simplified checkout/publish steps with the Git and Artifactory configuration established in earlier labs.

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { echo 'Checking out source code' } }
    stage('Build') { steps { sh 'mvn clean compile' } }
    stage('Test') { steps { sh 'mvn test' } }
    stage('Package') { steps { sh 'mvn package -DskipTests' } }
    stage('Publish') { steps { echo 'Publishing artifact to Artifactory' } }
    stage('Approval') { steps { input message: 'Proceed with deployment?', ok: 'Deploy' } }
    stage('Deploy') { steps { echo 'Simulating application deployment' } }
  }
  post {
    success { echo 'Pipeline completed successfully' }
    failure { echo 'Pipeline failed' }
    always { junit allowEmptyResults: true, testResults: 'target/surefire-reports/*.xml' }
  }
}
```

Capstone: Execute Successfully

First, achieve a fully green Pipeline run across all seven stages before introducing any failures.



Deliberately Break the Pipeline

Introduce a controlled failure to practice real-world troubleshooting. Observe which stage fails and what error appears in the Console Output.

Fail a Unit Test

Modify a test assertion so it produces a test failure.

Use an Incorrect Branch

Reference a branch name that does not exist in the repository.

Change an Artifact Path

Point the publish step to a path that does not exist.

Wrong Credential ID

Reference a credential ID that is not configured in Jenkins.

Troubleshoot the Failure

Use all available diagnostic tools to identify the root cause of the failure you introduced.



Pipeline Visualization

Identify which stage turned red and which stages were skipped as a result.



Console Output

Find the first meaningful error message — not just the final FAILURE line.

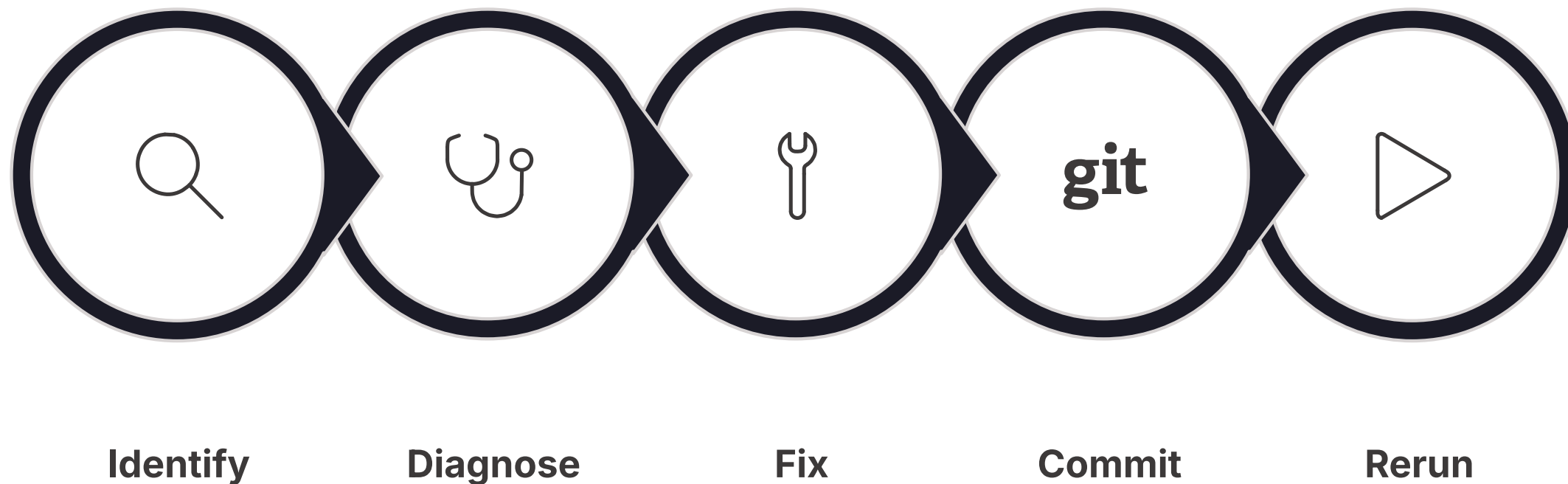


Test Results

Review the JUnit test report for specific assertion failures and stack traces.

Fix & Rerun

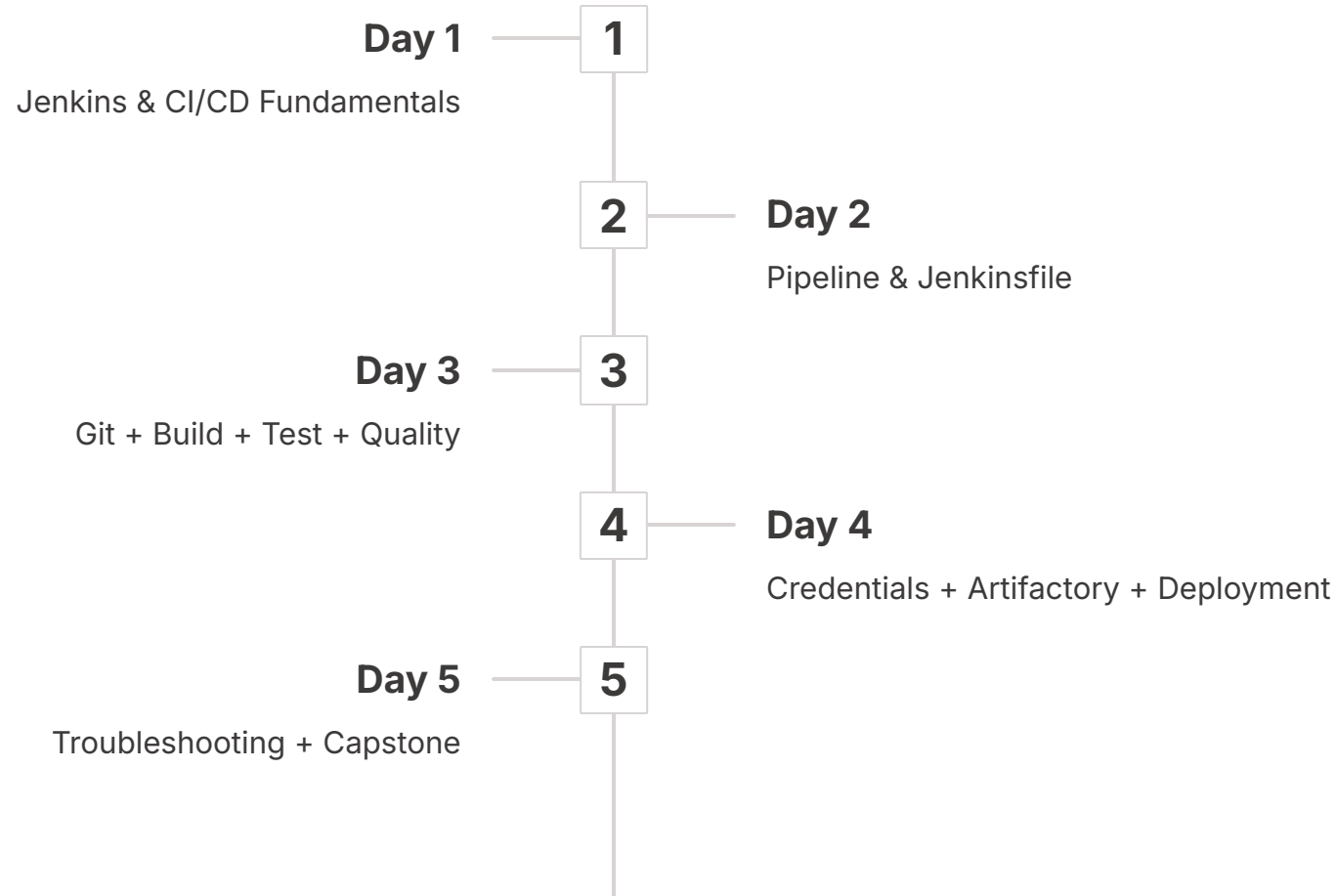
The objective is not merely to make the Pipeline green. Learners should be able to **explain why it failed and why their fix worked.**



Capstone Deliverables

1	Working Jenkinsfile A complete, valid Jenkinsfile committed to the repository.
2	Successful Pipeline All stages — Checkout → Build → Test → Package — pass cleanly.
3	Automated Build & Tests Maven build and unit tests run automatically on each trigger.
4	Artifact Published Jenkins publishes the packaged artifact to JFrog Artifactory.
5	Approval + Simulated Deploy Manual approval gate followed by a successful simulated deployment.
6	Troubleshooting Evidence Break → Diagnose → Fix → Rerun cycle demonstrated and explained.

Five-Day Learning Journey



From *"I can create a Jenkins job."* → To *"I can build, understand, execute, and troubleshoot a basic end-to-end Jenkins CI/CD Pipeline."*